PRACTICAL-10

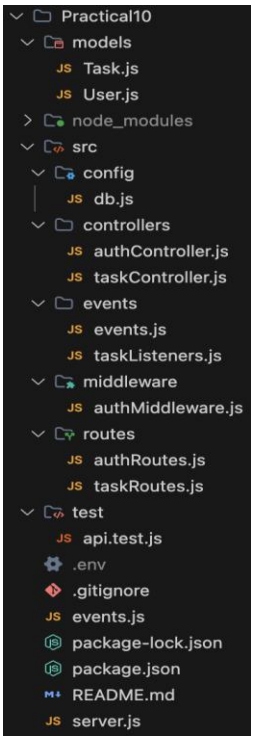
PROBLEM DEFINITION:

• Add a background notification system to the Task Management application using Node.js built-in EventEmitter.
• When a new task is created via the API, emit a task-created event handled asynchronously.
• The event handler must log a notification (task title, timestamp, assigned user) without blocking the main response.
• The main API response must return immediately, before the event handler finishes executing.
• Demonstrate the difference between synchronous and asynchronous handling by logging timestamps at both the API response point and the event handler execution point.

SUPPLEMENTARY PROBLEMS:

• Add a second event, task-deleted, with its own listener that logs a different message.
• Add an error event listener that catches and logs any error thrown inside the task-created handler.
• Simulate a slow notification handler (e.g., setTimeout of 2 seconds) and confirm the API still responds instantly.

ARCHITECTURE / DIAGRAM:

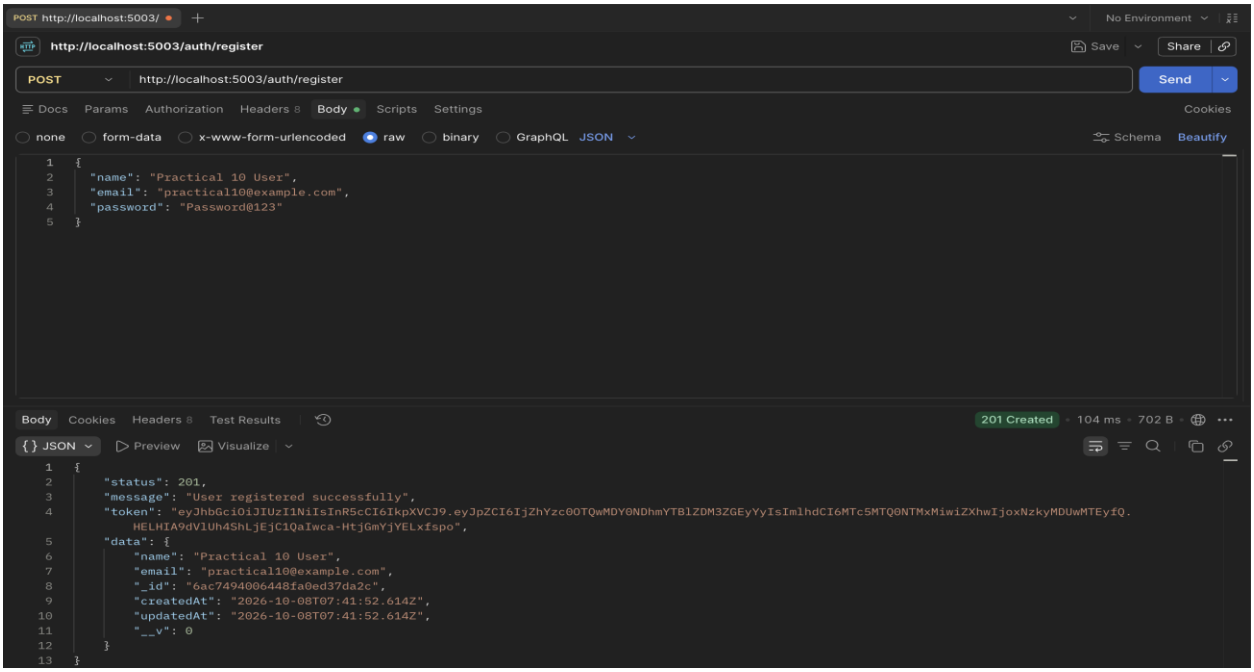


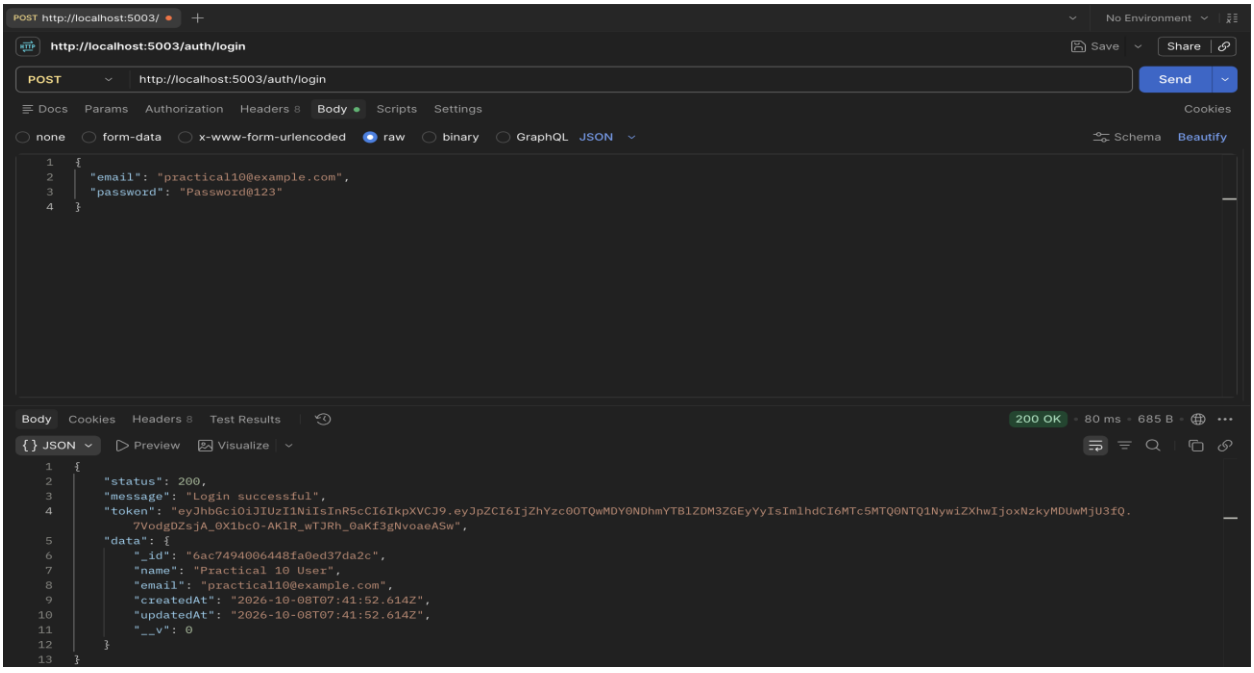
OUTPUT (SUPPLEMENTARY PROBLEMS INCLUDED):

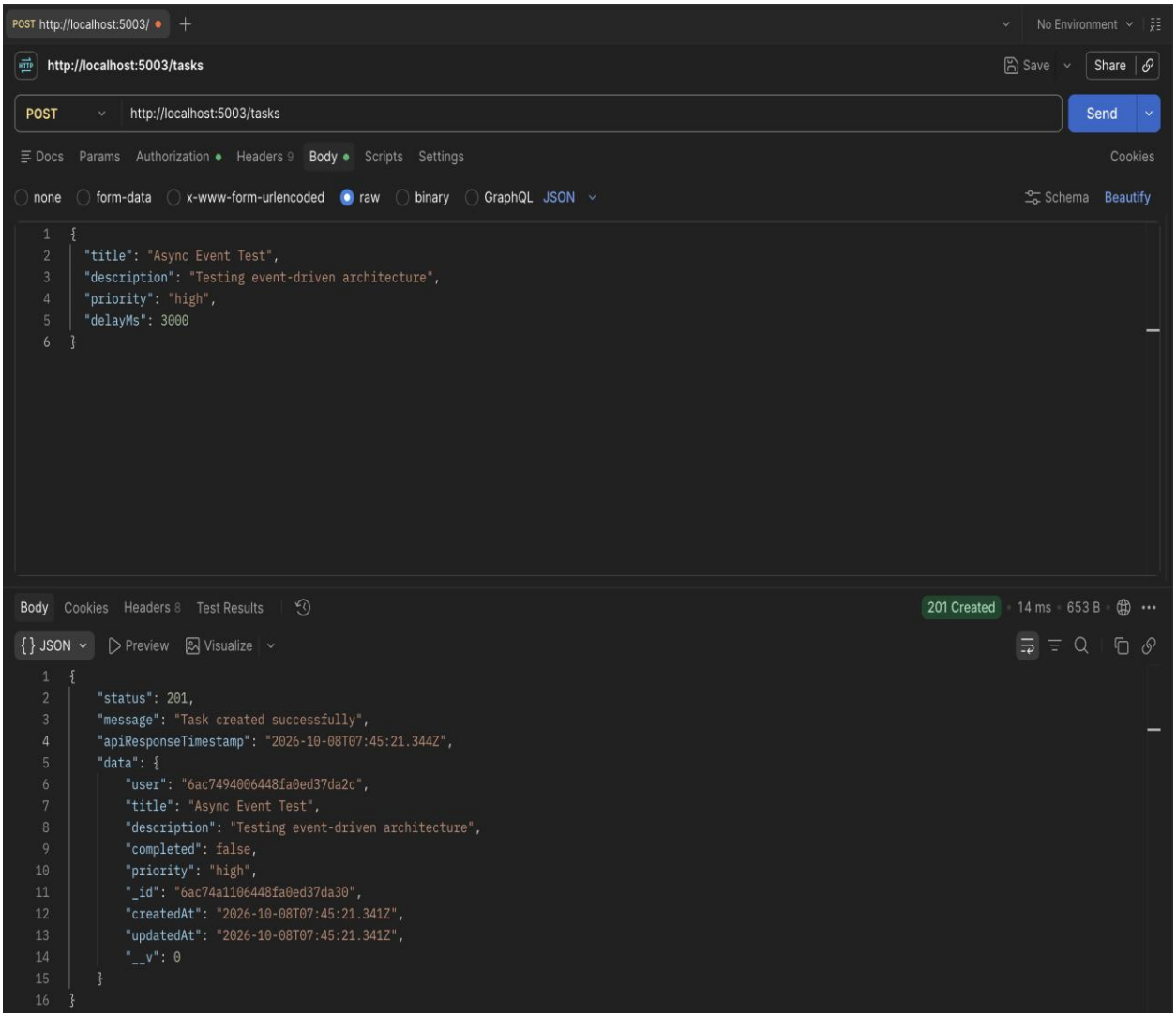
```
kamanijainamrajeshkumar@Kamanis-MacBook-Air Practicals % cd Practical10
kamanijainamrajeshkumar@Kamanis-MacBook-Air Practical10 % npm start

> practical10-event-driven-architecture@1.0.0 start
> node server.js

✅ Connected to MongoDB: localhost/practical10_events_db
================================================================================
🚀 Practical 10: Event-Driven Architecture API running on port 5003
🔑 Auth Endpoint: http://localhost:5003/auth
⚡ Task API Endpoint: http://localhost:5003/tasks
================================================================================
```



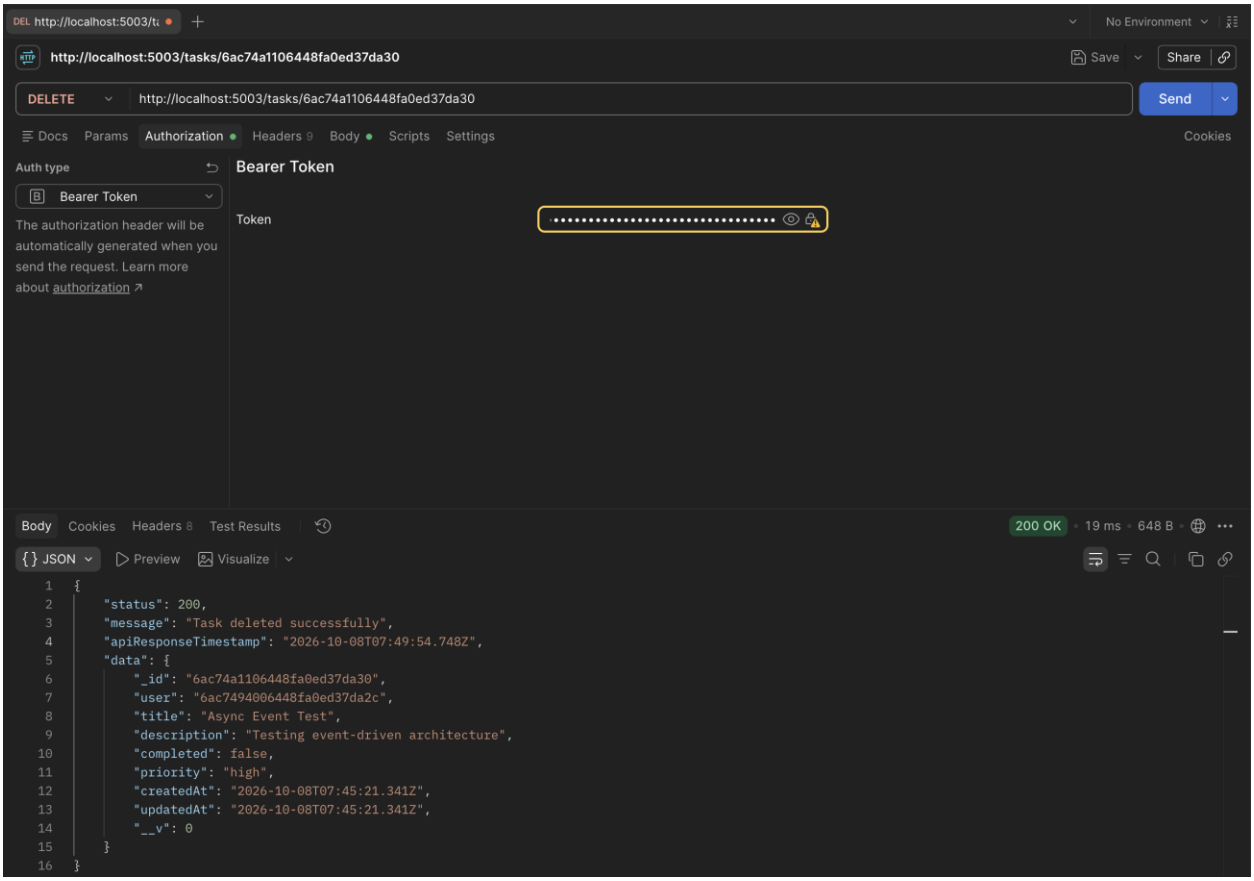





```
kamanijainamrajeshkumar@Kamanis-MacBook-Air Practicals % cd Practical10
kamanijainamrajeshkumar@Kamanis-MacBook-Air Practical10 % npm start

> practical10-event-driven-architecture@1.0.0 start
> node server.js

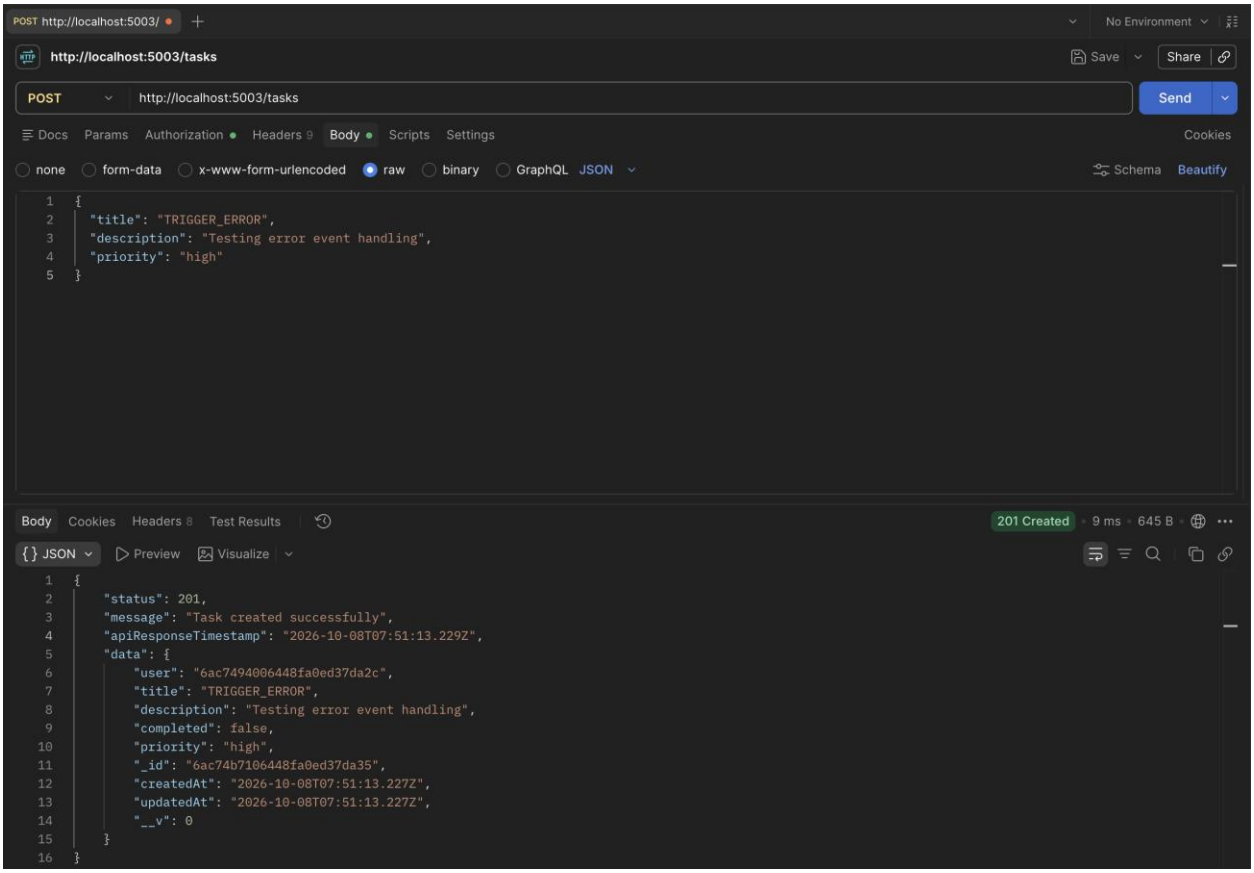
🍃 Connected to MongoDB: localhost/practical10_events_db
================================================================================
🚀 Practical 10: Event-Driven Architecture API running on port 5003
🔑 Auth Endpoint: http://localhost:5003/auth
⚡ Task API Endpoint: http://localhost:5003/tasks
================================================================================
[2026-10-08T07:40:12.617Z] POST /auth/login 400 - 13ms
[2026-10-08T07:41:17.664Z] POST /auth/registe 404 - 2ms
[2026-10-08T07:41:34.319Z] POST /auth/registe 404 - 1ms
[2026-10-08T07:41:52.624Z] POST /auth/register 201 - 103ms
[2026-10-08T07:44:17.791Z] POST /auth/login 200 - 76ms
[API] Response sent at 2026-10-08T07:45:21.344Z
[Notification Listener START] Event triggered for Task "Async Event Test" at 2026-10-08T07:45:
21.345Z
[2026-10-08T07:45:21.345Z] POST /tasks 201 - 13ms
[Notification Listener FINISHED] Task "Async Event Test" (ID: 6ac74a1106448fa0ed37da30, User:
6ac7494006448fa0ed37da2c) processed background notification at 2026-10-08T07:45:24.345Z
```




```
kamanijainamrajeshkumar@Kamanis-MacBook-Air Practicals % cd Practical10
kamanijainamrajeshkumar@Kamanis-MacBook-Air Practical10 % npm start

> practical10-event-driven-architecture@1.0.0 start
> node server.js

🍃 Connected to MongoDB: localhost/practical10_events_db
================================================================================
🚀 Practical 10: Event-Driven Architecture API running on port 5003
🔑 Auth Endpoint: http://localhost:5003/auth
⚡ Task API Endpoint: http://localhost:5003/tasks
================================================================================
[2026-10-08T07:40:12.617Z] POST /auth/login 400 - 13ms
[2026-10-08T07:41:17.664Z] POST /auth/registe 404 - 2ms
[2026-10-08T07:41:34.319Z] POST /auth/registe 404 - 1ms
[2026-10-08T07:41:52.624Z] POST /auth/register 201 - 103ms
[2026-10-08T07:44:17.791Z] POST /auth/login 200 - 76ms
[API] Response sent at 2026-10-08T07:45:21.344Z
[Notification Listener START] Event triggered for Task "Async Event Test" at 2026-10-08T07:45:
21.345Z
[2026-10-08T07:45:21.345Z] POST /tasks 201 - 13ms
[Notification Listener FINISHED] Task "Async Event Test" (ID: 6ac74a1106448fa0ed37da30, User:
6ac7494006448fa0ed37da2c) processed background notification at 2026-10-08T07:45:24.345Z
[API] Delete response sent at 2026-10-08T07:49:54.748Z
[Deletion Listener START] Event triggered for deleted Task "Async Event Test" at 2026-10-08T07
:49:54.749Z
[2026-10-08T07:49:54.749Z] DELETE /tasks/6ac74a1106448fa0ed37da30 200 - 18ms
[Deletion Listener FINISHED] Task "Async Event Test" cleanup notification recorded at 2026-10-
08T07:49:55.750Z
```




```
kamanijainamrajeshkumar@Kamanis-MacBook-Air Practicals % cd Practical10
kamanijainamrajeshkumar@Kamanis-MacBook-Air Practical10 % npm start

> practical10-event-driven-architecture@1.0.0 start
> node server.js

🌿 Connected to MongoDB: localhost/practical10_events_db
======================================================================
🚀 Practical 10: Event-Driven Architecture API running on port 5003
🔑 Auth Endpoint: http://localhost:5003/auth
⚡ Task API Endpoint: http://localhost:5003/tasks
======================================================================
[2026-10-08T07:40:12.617Z] POST /auth/login 400 - 13ms
[2026-10-08T07:41:17.664Z] POST /auth/registe 404 - 2ms
[2026-10-08T07:41:34.319Z] POST /auth/registe 404 - 1ms
[2026-10-08T07:41:52.624Z] POST /auth/register 201 - 103ms
[2026-10-08T07:44:17.791Z] POST /auth/login 200 - 76ms
[API] Response sent at 2026-10-08T07:45:21.344Z
[Notification Listener START] Event triggered for Task "Async Event Test" at 2026-10-08T07:45:
21.345Z
[2026-10-08T07:45:21.345Z] POST /tasks 201 - 13ms
[Notification Listener FINISHED] Task "Async Event Test" (ID: 6ac74a1106448fa0ed37da30, User:
6ac7494006448fa0ed37da2c) processed background notification at 2026-10-08T07:45:24.345Z
[API] Delete response sent at 2026-10-08T07:49:54.748Z
[Deletion Listener START] Event triggered for deleted Task "Async Event Test" at 2026-10-08T07
:49:54.749Z
[2026-10-08T07:49:54.749Z] DELETE /tasks/6ac74a1106448fa0ed37da30 200 - 18ms
[Deletion Listener FINISHED] Task "Async Event Test" cleanup notification recorded at 2026-10-
08T07:49:55.750Z
[API] Response sent at 2026-10-08T07:51:13.229Z
[Notification Listener START] Event triggered for Task "TRIGGER_ERROR" at 2026-10-08T07:51:13.
229Z
[Error Listener CAUGHT] ❌ Event Handler Error: Failed to deliver notification for Task "TRIGG
ER_ERROR": Invalid recipient
[2026-10-08T07:51:13.230Z] POST /tasks 201 - 8ms
```

KEY QUESTIONS / ANALYSIS:

1) Why does emitting an event not block the API response, even though both run in the same Node.js process?

EventEmitter calls the registered listener synchronously by default, but asynchronous operations inside the listener, such as setTimeout(), allow the API response to return without waiting for those operations to finish. Therefore, the API can respond immediately while the notification completes later.

2) What would happen to API response time if the notification logic were placed directly inside the POST route instead of in an event handler?

If the notification logic were executed directly inside the POST route and involved slow operations, the API response would be delayed until those operations finished. Using an event handler separates the notification logic from the main route, helping the API respond faster.

3) Why is EventEmitter a reasonable choice for this scale of application, but not for a production system handling millions of events per day?

EventEmitter is simple, lightweight, and suitable for small applications running in a single Node.js process. However, it does not provide persistent message storage or built-in delivery guarantees. For large production systems, message brokers such as Apache Kafka or RabbitMQ are more suitable because they support scalable event processing and reliable message delivery.

GITHUB REPOSITORY:

https://github.com/9081495207/ITUE301-Advanced-Web-Development-Frameworks